

How to build your own **green** observability pipeline



Presented by Diana Todea



Whoami

HEAD OF DEVREL @VictoriaMetrics

CNCF Ambassador, TAG Lead Operational Resilience, OpenTelemetry Docs Approver

LinkedIn: <https://www.linkedin.com/in/diana-todea-b2a79968/>

GitHub: didiViking

Bluesky: @didiViking.bsky.social

X: @dianavtodea

Mastodon: @dianatodea

Instagram: @didiVikingdevrel

Medium: @dianatodea



The motivation



Participate at a conference, saw a talk on [SPRUCE](#) project



Decided to fiddle with the project and integrate it with an observability backend and frontend

The problem

- 🌱 Cost and sustainability data often live in reports, not in engineering workflows
- 🌱 Observability has an environmental footprint
- 🌱 More data $\neq$ better observability



The objective

 Cloud cost is telemetry. Sustainability is telemetry

 We should be able to build viable Green Observability pipeline and observe our GreenOps data

Building the pipeline

 <https://github.com/didiViking/greenops-observability-reference-architecture>

 An open source green pipeline using Azure FOCUS, SPRUCE, Python, VictoriaMetrics, Grafana and Prometheus exposition format



Architecture overview



Why SPRUCE?

 <https://github.com/digitalpebble/spruce>

 My project separates billing normalization from observability

 SPRUCE bridges that gap: it reads Azure FOCUS exports, normalizes the data into a FOCUS-compatible Parquet dataset and enriches each billing record with sustainability related attributes derived from open environmental models.

Why VictoriaMetrics?

🌱 VictoriaMetrics is an OSS project, which provides a lightweight, high-performance time-series database that is fully compatible with Prometheus ecosystem and requires very little operational overhead.



🌱 I'll use vmui (VictoriaMetrics UI) to:

- 🌱 discover metrics
- 🌱 inspect labels
- 🌱 experiment with PromQL
- 🌱 explore metric cardinality
- 🌱 validate the exported sustainability metrics

🌱 <https://github.com/VictoriaMetrics>

The VictoriaMetrics Stack - Full Observability

Metrics + Logs + Traces: one consistent and efficient solution

VictoriaMetrics

Metrics

Time series DB
Prometheus-compatible
MetricsQL

VictoriaLogs

Logs

Fast logs database
LogsQL query language
Schema-less

VictoriaTraces

Traces

Distributed tracing
OpenTelemetry native
10× compression

Unified on a single, coherent, cost-efficient platform · Open Source · Enterprise · Cloud

Execution step 1: Export billing data

 Export Azure FOCUS data

Cost Management: Virginia Diana Todea | Exports ... [What are my top 5 services with highest cost](#) [What's my current month's cost by Subscription?](#) +1

Billing account

Search « [+ Create](#) [Refresh](#) | [Run now](#) [Enable](#) [Disable](#) [Delete](#) [Feedback](#) [Help](#) ▾

Scope: [Virginia Diana Todea](#)

[You can now export to storage accounts with firewalls. Learn more](#)

Name	Type	Schedule status	Dataset version	Last run	Frequency
☐ Daily-actual-cost	Cost and usage details (actual)	Active	2021-10-01	7/9/2026, 9:13 AM UTC	Daily
☐ Daily_Focus-focus-cost	Cost and usage details (FOCUS) - Preview	Active	1.0r2	7/9/2026, 5:13 AM UTC	Daily

Execution step 2: Normalize and enrich with SPRUCE

- 🌱 Execute SPRUCE against the generated input file

```
docker run \  
-i /data/focus.csv  
-f FOCUS
```

- 🌱 SPRUCE produces a normalized Parquet dataset similar to

```
exports/cost-reports/result-focus
```

Execution step 3: Convert data into metrics



Install and run VictoriaMetrics, open vmui



Export Prometheus metrics (convert the SPRUCE Parquet dataset into Prometheus metrics)

Execution step 4: Store and query

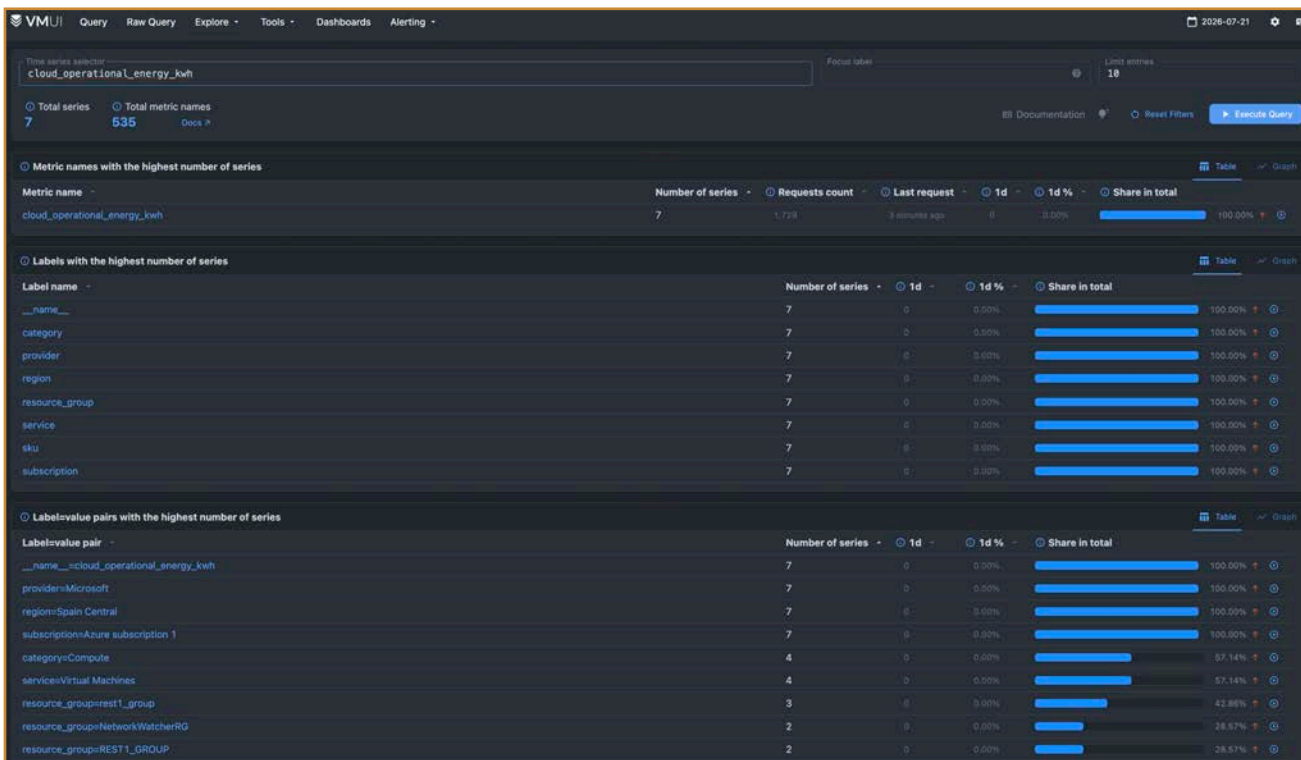
 Install and run Grafana OSS, configure Prometheus datasource

 Import Grafana dashboard using specific carbon related metrics

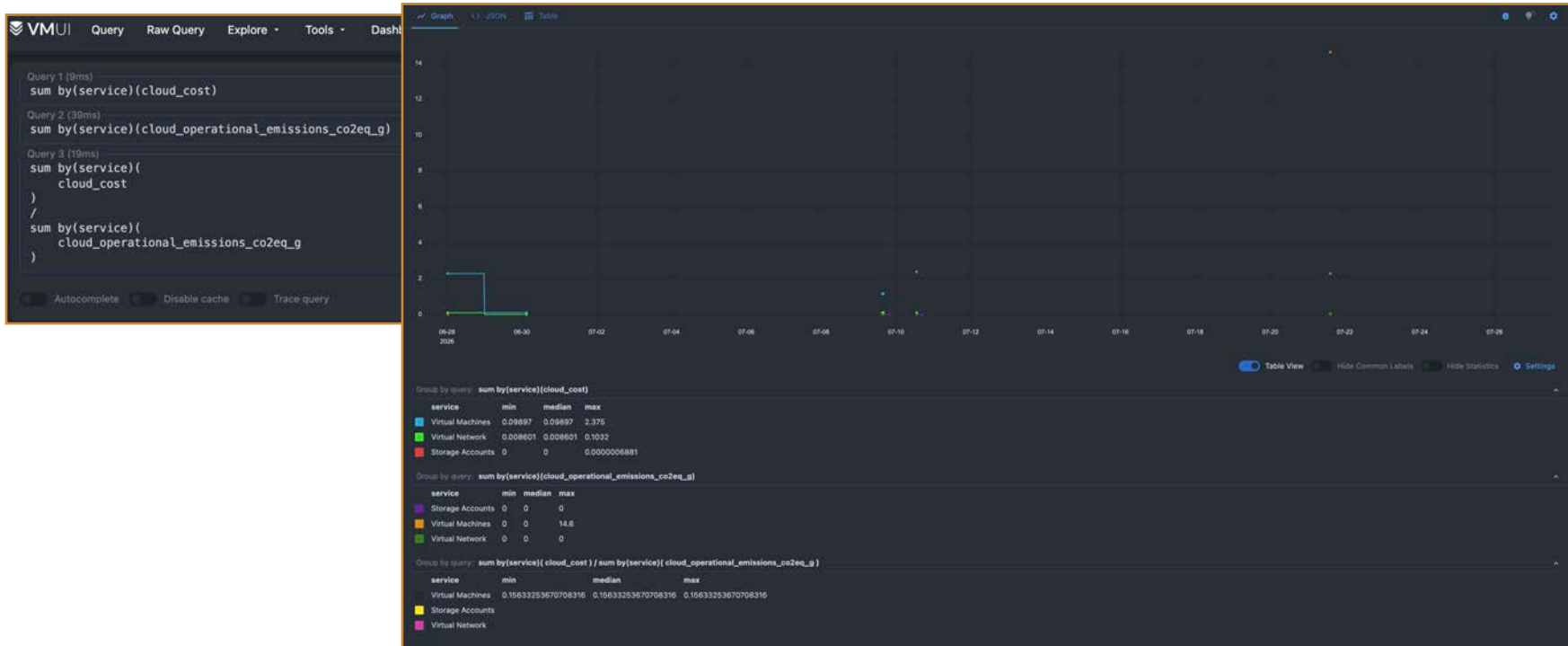
Exported metrics

	Metric Name	Description
1	 cloud_cost	Monetary cost for each billing record
2	 cloud_usage_quantity	Resource consumption quantity
3	 cloud_operational_energy_kwh	Estimated operational energy consumption
4	 cloud_operational_emissions_co2eq_g	Operational CO ₂ emissions
5	 cloud_embodied_emissions_co2eq_g	Embodied carbon emissions
6	 cloud_carbon_intensity	Carbon intensity associated with the region
7	 cloud_power_usage_effectiveness	Data center Power Usage Effectiveness (PUE)
8	 cloud_water_usage_effectiveness	Water Usage Effectiveness (WUE)
9	 cloud_water_cooling_l	Estimated cooling water consumption
10	 cloud_water_electricity_production_l	Water consumed during electricity production
11	 cloud_water_consumption_stress_area_l	Water consumption in stressed regions
12	 cloud_resource_info	Inventory metric exposing cloud resources as Prometheus labels
13	 cloud_meter_info	Inventory metric exposing Azure billing meters, categories, and subcategories as labels

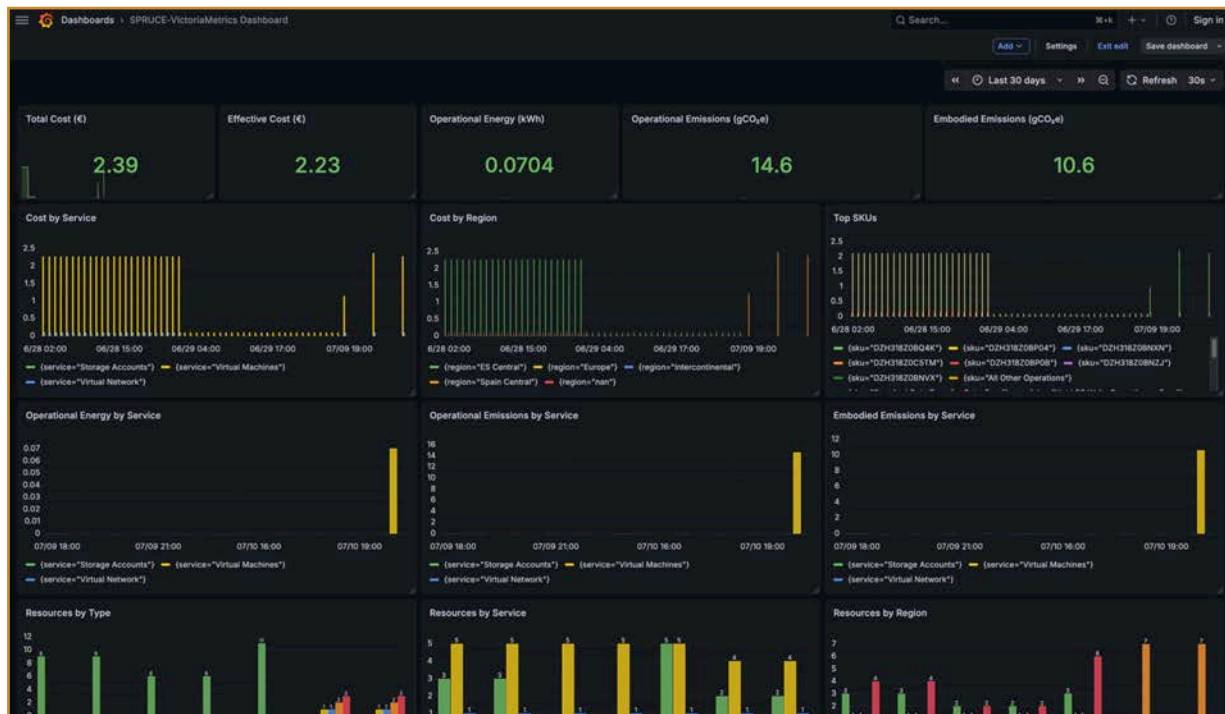
Explore metrics in vmui



Explore metrics in vmui



Execution step 5: visualize your GreenO11y pipeline



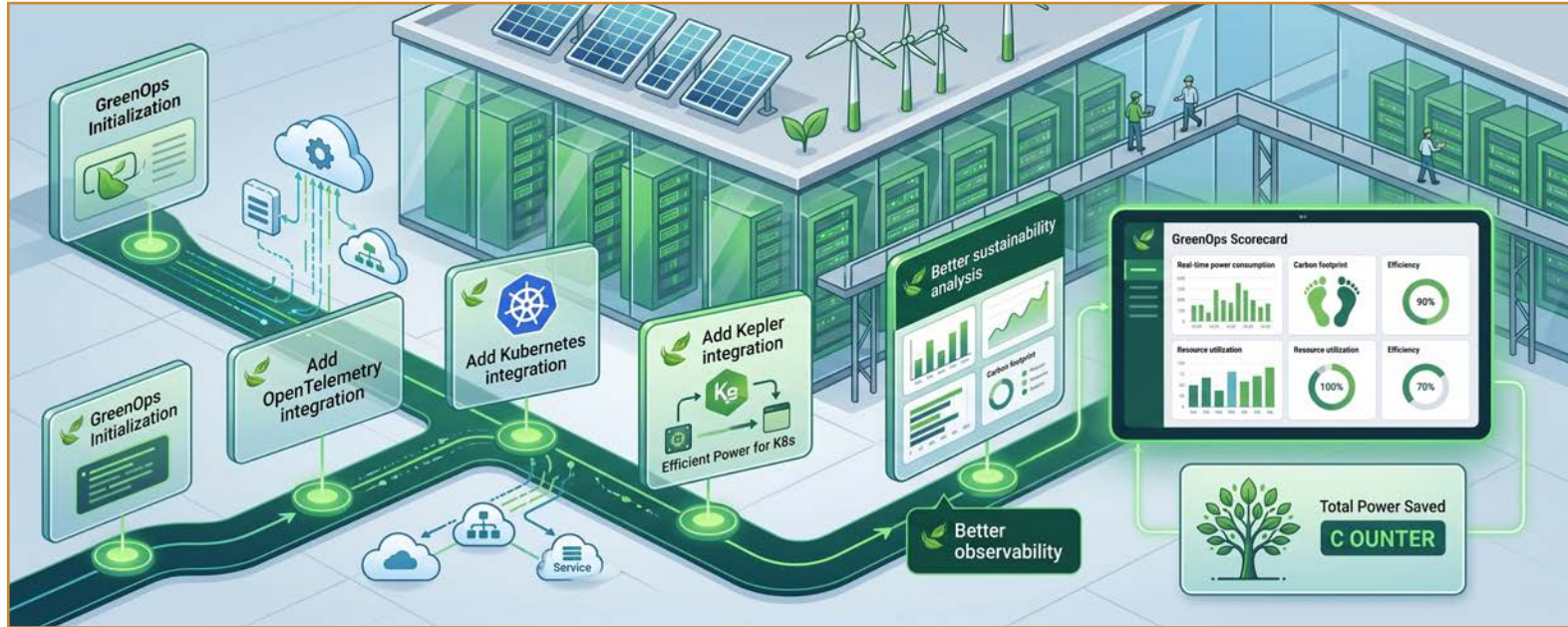
What didn't go as expected

- 🌱 When I first created my pipeline, I created a data simulator, which added more complexity
- 🌱 I used Azure instead of AWS reports, due to some AWS free tier limitations
- 🌱 Found out that the Azure Billing had 2 types of reports and only the legacy one was supported by SPRUCE

Working with maintainers

- 🌱 Created a bug in SPRUCE project: <https://github.com/DigitalPebble/spruce/issues/246>
- 🌱 Collaboration with SPRUCE maintainer
- 🌱 Added new support for Azure FOCUS 1.0 and improve documentation

Future improvements



Take-aways

- 🌱 Think about a concrete outcome and use case
- 🌱 Start simple and stick with OSS or cloud native tools (better collaboration and faster feedback)
- 🌱 Use a storage backend for your data and a frontend for visualization
- 🌱 If stuck, think if there are any alternatives already available

Resources

1

<https://github.com/didiViking/greenops-observability-reference-architecture>

2

<https://github.com/digitalpebble/spruce>

3

<https://github.com/VictoriaMetrics>

4

<https://medium.com/@dianatodea/building-a-greenops-observability-pipeline-from-spruce-to-carbon-aware-metrics-with-open-source-c7e496dbc236>

5

<https://medium.com/@dianatodea/observability-has-a-data-hoarding-problem-70df85c88b9f>



Join us on Slack!

Thank you!



Presented by Diana Todea

LinkedIn: <https://www.linkedin.com/in/diana-todea-b2a79968/>

GitHub: didiViking

Bluesky: @didiviking.bsky.social

X: @dianavtodea

Mastodon: @dianatodea

Instagram: @didivikingdevrel

Medium: @dianatodea

